Тестовое задание на стажировку DevOps

СОДЕРЖАНИЕ

СОДЕРЖАНИЕ 2
ВВЕДЕНИЕ..... 3
ОСНОВНАЯ ЧАСТЬ..... 4
ЗАКЛЮЧЕНИЕ 10

ВВЕДЕНИЕ

Работу над проектом я выполнял в изолированном LXC-контейнере на базе гипервизора Proxmox с установленной ОС Ubuntu 24.04. Для обеспечения переносимости, изоляции процессов и соответствия современным DevOps-практикам я принял решение не устанавливать веб-сервер напрямую в систему, а сразу реализовать всю инфраструктуру с использованием Docker.

Ниже описан поэтапный процесс развертывания и настройки сервиса.

ОСНОВНАЯ ЧАСТЬ

Этап 1: Подготовка конфигурации Nginx

Цель: Создать базовую структуру для контейнера и настроить прокси-сервер.

Я создал директорию для проекта и подготовил базовые файлы Dockerfile для сборки образа на базе nginx:1.29.6 и пользовательский конфигурационный файл nginx.conf (см. Рисунок 1), в котором переопределил стандартные настройки.

```
 GNU nano 7.2                                                    nginx.conf
server {
    listen 80;
    server_name localhost;
    root /usr/share/nginx/html;
    index index.html;
    location / {
        ssi on;
    }
}
```

Рисунок 1. Первоначальная настройка nginx.conf

Этап 2: Создание статической страницы

Цель: Выполнить требование по выводу статического текста 'Hello

Что было сделано: Был создан файл index.html (см. Рисунок 2 и Рисунок директорию отдачи статики Nginx (/usr/share/nginx/html).

```
 GNU nano 7.2                                                    index.html
<!DOCTYPE html>
<html>
<head>
    <title>Test Task</title>
</head>
<body>
    <h1>Hello DevOps world!</h1>
</body>
</html>
```

Рисунок 2. Index.html с статическим текстом

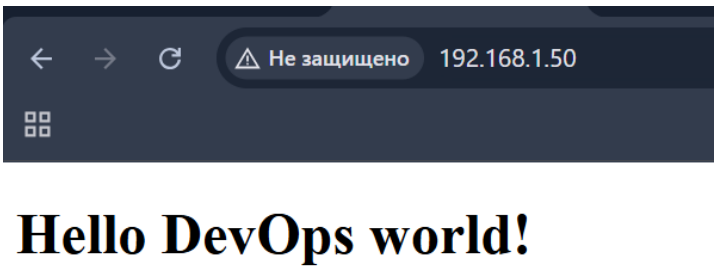

Рисунок 3. Проверка работоспособности сайта

Этап 3: Настройка сетевой безопасности (UFW)

Цель: Ограничить доступ к серверу, оставив только необходимые для работы порты.

Что было сделано: На сервере был настроен и активирован межсетевой экран UFW. Я явно разрешил подключения по протоколам SSH, HTTP и , после чего заблокировал весь остальной входящий трафик (см. Рисунок 4 и Рисунок 5).

```
root@test-task-simbirsoft:~# sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), deny (routed)
New profiles: skip

To                         Action      From
--                         ------      ----
22/tcp                     ALLOW IN    Anywhere
80/tcp                     ALLOW IN    Anywhere
443/tcp                    ALLOW IN    Anywhere
22/tcp (v6)                ALLOW IN    Anywhere (v6)
80/tcp (v6)                ALLOW IN    Anywhere (v6)
443/tcp (v6)               ALLOW IN    Anywhere (v6)
```

Рисунок 4. Проверка доступных портов на сервере

```
ittori@DESKTOP-CKH6FFJ:~/.ssh$ nmap -p 22,80,443,9090,9999,5432,53,51820 192.168.1.50
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-03-25 16:08 MSK
Nmap scan report for test-task-simbirsoft.lan (192.168.1.50)
Host is up (0.0054s latency).

PORT      STATE    SERVICE
22/tcp    open     ssh
53/tcp    filtered domain
80/tcp    open     http
443/tcp   open     https
5432/tcp  filtered postgresql
9090/tcp  filtered zeus-admin
9999/tcp  filtered abyss
51820/tcp filtered unknown

Nmap done: 1 IP address (1 host up) scanned in 1.23 seconds
ittori@DESKTOP-CKH6FFJ:~/.ssh$
```

Рисунок 5. Проверка доступных портов сервера с хоста

Этап 4: Динамическое время

Цель: Выводить на странице актуальное время с задержкой не более 1 минуты.

Что было сделано: Для динамического отображения времени без использования сложных backend-языков я применил технологию SSI (Server Side Includes). В nginx.conf была добавлена директива *ssi on*;, а в index.html команда для вывода переменной времени (см. Рисунок 6). Для корректного отображения времени в Dockerfile был задан локальный часовой пояс *ENV*

```
  GNU nano 7.2                                          index.html
<!DOCTYPE html>
<html>
<head>
    <title>Test Task</title>
</head>
<body>
    <h1>Hello DevOps world!</h1>
    <p>Time: <!--# echo var="date_local" --></p>
</body>
</html>
```

Рисунок 6. Index.html с отображением времени

Этап 5: Шифрование трафика (SSL/HTTPS)

Цель: Обеспечить доступ к странице по защищенному порту 443.

Что было сделано: С помощью утилиты openssl я сгенерировал самоподписанный сертификат (RSA 2048). Затем я обновил Dockerfile, добавив копирование этих ключей внутрь образа, и прописал пути к ним в блоке server файла nginx.conf, настроив прослушивание порта 443 (см. Рисунок 7).

Рисунок 7. Проверка https подключения

Этап 6: Автоматизация резервного копирования

Цель: Настроить регулярный бэкап настроек по расписанию.

Что было сделано: Я создал директорию backups и написал Bash-скрипт .tar.gz, динамически подставляя дату и время создания в имя файла. Скрипту были выданы права на исполнение. Для автоматизации я добавил задачу в системный планировщик cron, настроив запуск на 01:00 UTC каждый понедельник (см. Рисунок 9).

```
  GNU nano 7.2                                                    backup.sh
#!/bin/bash

BACKUP_DIR="/root/nginx-project/backups"
SOURCE_DIR="/root/nginx-project"
DATE=$(date +"%Y-%m-%d_%H-%M-%S")
ARCHIVE_NAME="nginx_backup_$DATE.tar.gz"
tar -czf "$BACKUP_DIR/$ARCHIVE_NAME" "$SOURCE_DIR"
echo "Create backup: $BACKUP_DIR/$ARCHIVE_NAME"
```

Рисунок 8. Содержимое файла backup.sh

```
  GNU nano 7.2                                  /tmp/crontab.faKkFw/crontab
# Edit this file to introduce tasks to be run by cron.
#
# Each task to run has to be defined through a single line
# indicating with different fields when the task will be run
# and what command to run for the task
#
# To define the time you can provide concrete values for
# minute (m), hour (h), day of month (dom), month (mon),
# and day of week (dow) or use '*' in these fields (for 'any').
#
# Notice that tasks will be started based on the cron's system
# daemon's notion of time and timezones.
#
# Output of the crontab jobs (including errors) is sent through
# email to the user the crontab file belongs to (unless redirected).
#
# For example, you can run a backup of all your user accounts
# at 5 a.m every week with:
# 0 5 * * 1 tar -zcf /var/backups/home.tgz /home/
#
# For more information see the manual pages of crontab(5) and cron(8)
#
# m h  dom mon dow   command
0 1 * * 1 /root/nginx-project/backup.sh
```

Рисунок 9. Добавление задачи в cron

Этап 7: Работа контейнера не от root

Цель: Запустить Nginx от имени непривилегированного пользователя.

Что было сделано: По умолчанию Nginx запускает master-процесс от root. Для соблюдения принципа наименьших привилегий я модифицировал

рекурсивно выданы права владения (chown) на все необходимые рабочие директории (кэш, логи, конфиги, pid-файл). Директива USER nginx переключила контекст, гарантируя, что внутри контейнера сервис не имеет root-прав (см. Рисунок 11).

```
RUN chown -R nginx:nginx /usr/share/nginx/html && \
    chown -R nginx:nginx /etc/nginx/conf.d && \
    chown -R nginx:nginx /etc/nginx/ssl && \
    chown -R nginx:nginx /var/cache/nginx && \
    chown -R nginx:nginx /var/log/nginx && \
    touch /var/run/nginx.pid && \
    chown -R nginx:nginx /var/run/nginx.pid

USER nginx
```

Рисунок 10. Добавление блоков в Dockerfile

```
root@test-task-simbirsoft:~/nginx-project# docker exec my-nginx whoami
nginx
root@test-task-simbirsoft:~/nginx-project#
```

Рисунок 11. Проверка проделанной работы

Этап 8: Безопасный SSH-доступ

Цель: Настроить авторизацию на сервере по ключам, исключив использование паролей.

Что было сделано: На локальной машине был сгенерирован SSH-ключ подключения в файле ~/.ssh/config был создан алиас, позволяющий заходить на сервер короткой командой (см. Рисунок 12).

```
ittori@DESKTOP-CKH6FFJ:~/.ssh$ cat config
Host test-task
    HostName 192.168.1.50
    User root
    IdentityFile ~/.ssh/id_rsa
ittori@DESKTOP-CKH6FFJ:~/.ssh$ ssh test-task
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.17.2-1-pve x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro
Last login: Wed Mar 25 14:42:12 2026 from 192.168.1.150
root@test-task-simbirsoft:~#
```

Рисунок 12. Подключение к серверу по ssh без пароля

Этап 9: Итоговый Dockerfile

Цель: Собрать финальный образ со всеми внедренными практиками.

Что было сделано: Финальный Dockerfile объединил в себе все предыдущие шаги: установку таймзоны, копирование конфигураций и сертификатов, настройку прав доступа и переключение пользователя (см. Рисунок 13). После этого образ был успешно собран и запущен с пробросом портов.

```
  GNU nano 7.2                                          Dockerfile
FROM nginx:1.29.6
ENV TZ="Europe/Moscow"
RUN rm /etc/nginx/conf.d/default.conf
RUN mkdir -p /etc/nginx/ssl

COPY nginx.conf /etc/nginx/conf.d/nginx.conf
COPY index.html /usr/share/nginx/html/index.html

COPY nginx-selfsigned.crt /etc/nginx/ssl/
COPY nginx-selfsigned.key /etc/nginx/ssl/

RUN chown -R nginx:nginx /usr/share/nginx/html && \
    chown -R nginx:nginx /etc/nginx/conf.d && \
    chown -R nginx:nginx /etc/nginx/ssl && \
    chown -R nginx:nginx /var/cache/nginx && \
    chown -R nginx:nginx /var/log/nginx && \
    touch /var/run/nginx.pid && \
    chown -R nginx:nginx /var/run/nginx.pid

USER nginx
CMD ["nginx", "-g", "daemon off;"]
```

Рисунок 13. Итоговый Dockerfile

ЗАКЛЮЧЕНИЕ

В ходе работы были успешно реализованы все обязательные и дополнительные пункты тестового задания. В изолированном Docker-контейнере развернут веб-сервер Nginx 1.29.6, работающий от непривилегированного пользователя. Настроена отдача статической страницы с динамическим отображением времени сервера с помощью технологии SSI.

Обеспечена комплексная защита инфраструктуры: настроен файрволл UFW (порты 22, 80, 443), сгенерирован SSL-сертификат для доступа по HTTPS, а авторизация на сервере осуществляется по надежному SSH-ключу. Также настроено автоматическое еженедельное резервное копирование с помощью cron